

1. Objetivo

Mediante el desarrollo de este proyecto, usted aplicará los conceptos de programación en lenguaje ensamblador vistos en clase para implementar el algoritmo de cifrado de flujo ChaCha20. Para ello, usted deberá manejar la lógica de alto nivel en C, pero implementar las funciones críticas de generación de flujo de clave y cifrado en lenguaje ensamblador de RISC-V, particularmente haciendo uso del ISA rv32i.

2. Descripción General

La criptografía de software eficiente es un requisito fundamental en sistemas empotrados modernos, donde el hardware dedicado para aceleración criptográfica puede no estar disponible. El algoritmo ChaCha20, diseñado por Daniel J. Bernstein en 2008, es un cifrador de flujo de alto rendimiento adoptado en protocolos de seguridad ampliamente desplegados, incluyendo TLS 1.3, el protocolo WireGuard y OpenSSH.

A diferencia de algoritmos basados en cajas de sustitución (*S-boxes*) o transformaciones afines, ChaCha20 pertenece a la familia de diseños ARX (*Add-Rotate-XOR*): toda su lógica se reduce a sumas de 32 bits, rotaciones de bits y operaciones XOR. Esta característica lo hace especialmente adecuado para implementación en procesadores sin unidad de punto flotante ni instrucciones criptográficas dedicadas, como es el caso de un procesador RISC-V rv32i base.

En este proyecto se deberá implementar una aplicación completa que utilice ChaCha20 para cifrar y descifrar cadenas de texto en memoria, utilizando como base el entorno de desarrollo basado en **LiteX** y que se le ha brindado para este proyecto. La implementación debe separar claramente las responsabilidades: la inicialización del estado, la gestión de bloques y la interfaz con los datos se realizará en C, mientras que las operaciones criptográficas críticas se implementarán en ensamblador RISC-V rv32i.

3. Especificación

3.1. Algoritmo ChaCha20

ChaCha20 es un cifrador de flujo que opera generando bloques de **64 bytes** de flujo de clave (*keystream*) a partir de una clave de 256 bits, un nonce de 96 bits y un contador de bloque de 32 bits. El texto cifrado se obtiene aplicando XOR entre el texto plano y el flujo de clave generado; el descifrado es idéntico, aplicando el mismo XOR sobre el texto cifrado.

Estado interno

El estado de ChaCha20 es una matriz de 4×4 palabras de 32 bits (16 palabras, 512 bits en total), organizada de la siguiente manera:

cte[0]	cte[1]	cte[2]	cte[3]
key[0]	key[1]	key[2]	key[3]
key[4]	key[5]	key[6]	key[7]
counter	nonce[0]	nonce[1]	nonce[2]

Las cuatro constantes son los bytes ASCII de la cadena “expand 32-byte k” divididos en palabras de 32 bits en formato *little-endian*: 0x61707865, 0x3320646e, 0x79622d32, 0x6b206574. La clave ocupa ocho palabras (256 bits), el contador una palabra (32 bits), y el nonce las tres palabras restantes (96 bits).

Operación cuarto de ronda (*Quarter Round*)

La operación primitiva del algoritmo es el *quarter round*, que transforma cuatro palabras del estado aplicando una secuencia de operaciones ARX. En la Figura 1 se presentan las funciones de referencia del algoritmo:

Cifrado y descifrado

Para cifrar un mensaje, se genera el flujo de clave bloque a bloque (64 bytes por bloque, incrementando el contador en 1 entre bloques) y se aplica XOR con el texto plano. El descifrado es idéntico: generar el mismo flujo de clave y aplicar XOR sobre el texto cifrado recupera el mensaje original.

3.2. Arquitectura de la aplicación

Para este proyecto se deberá implementar una arquitectura de software que separe claramente las responsabilidades entre el código de alto nivel (C) y las operaciones críticas de cifrado (ensamblador RISC-V).

- **Capa de alto nivel (C):** Esta capa será responsable de la inicialización de la matriz de estado del algoritmo (clave, nonce, contador y constantes), la gestión del contador de bloque entre llamadas consecutivas, la recepción y despliegue de mensajes, el manejo de mensajes de longitud variable (múltiples bloques y bloque final parcial), y la presentación de resultados.
- **Capa de bajo nivel (ensamblador RISC-V):** Esta capa implementará las funciones `chacha20_block` y `chacha20_xor`, que realizarán respectivamente la generación del bloque de flujo de clave y la aplicación del cifrado XOR sobre los datos. Estas funciones deberán seguir estrictamente las convenciones de llamada RISC-V, siguiendo el ABI, y estar optimizadas para eficiencia en el uso de registros y memoria.

```

// Operacion primitiva: quarter round sobre 4 palabras de 32 bits
void quarter_round(uint32_t *a, uint32_t *b, uint32_t *c, uint32_t *d) {
    *a += *b;    *d ^= *a;    *d = ROTL32(*d, 16);
    *c += *d;    *b ^= *c;    *b = ROTL32(*b, 12);
    *a += *b;    *d ^= *a;    *d = ROTL32(*d, 8);
    *c += *d;    *b ^= *c;    *b = ROTL32(*b, 7);
}

// Funcion de bloque: genera 64 bytes de keystream a partir del estado
void chacha20_block(uint32_t state[16], uint32_t out[16]) {
    uint32_t working[16];
    memcpy(working, state, 64);           // copia de trabajo del estado

    for (int i = 0; i < 10; i++) {        // 10 doble-rondas = 20 rondas
        // Rondas de columnas
        quarter_round(&working[0], &working[4], &working[ 8], &working[12]);
        quarter_round(&working[1], &working[5], &working[ 9], &working[13]);
        quarter_round(&working[2], &working[6], &working[10], &working[14]);
        quarter_round(&working[3], &working[7], &working[11], &working[15]);
        // Rondas diagonales
        quarter_round(&working[0], &working[5], &working[10], &working[15]);
        quarter_round(&working[1], &working[6], &working[11], &working[12]);
        quarter_round(&working[2], &working[7], &working[ 8], &working[13]);
        quarter_round(&working[3], &working[4], &working[ 9], &working[14]);
    }

    for (int i = 0; i < 16; i++)
        out[i] = working[i] + state[i];    // suma final del estado original
}

```

Figura 1: Funciones de referencia para ChaCha20: *quarter round* y función de bloque.

3.3. Requisitos específicos

3.3.1. Archivos en ensamblador RISC-V

Se deberán desarrollar **dos archivos** en ensamblador RISC-V:

- `chacha20_block.s` — Implementa la función `chacha20_block(state, output)`:
 - Recibe un puntero al arreglo de estado de 16 palabras de 32 bits y un puntero al búfer de salida de 16 palabras.
 - Realiza una copia de trabajo del estado y aplica las 20 rondas de transformaciones (10 doble-rondas compuestas de *quarter rounds* sobre columnas y diagonales, según la Figura 1).
 - Suma el estado original a la copia de trabajo transformada palabra por palabra.
 - Escribe el resultado (64 bytes de flujo de clave) en el búfer de salida.
 - Debe seguir estrictamente las convenciones de llamada RISC-V para parámetros (a0-a7), dirección de retorno (ra), y preservación de registros (s0-s11).

- `chacha20_xor.s` — Implementa la función `chacha20_xor(keystream, input, output, length)`:
 - Recibe punteros al flujo de clave generado, al búfer de entrada (texto plano o cifrado), al búfer de salida, y la longitud en bytes a procesar.
 - Aplica XOR byte a byte entre el flujo de clave y los datos de entrada, almacenando el resultado en el búfer de salida.
 - Debe operar correctamente para longitudes arbitrarias, incluyendo el bloque final parcial cuando el mensaje no es múltiplo de 64 bytes.
 - Debe seguir las convenciones de llamada RISC-V.

3.3.2. Programa principal en C

- El archivo `chacha20.c` deberá estar escrito en C y proporcionar la estructura de alto nivel de la aplicación.
- Será responsable de construir la matriz de estado inicial de 16 palabras a partir de las cuatro constantes del algoritmo, la clave de 256 bits (ocho palabras de 32 bits), el nonce de 96 bits (tres palabras de 32 bits) y el contador inicial.
- Deberá gestionar el procesamiento de mensajes de longitud variable, incrementando el contador de bloque en 1 entre llamadas consecutivas a `chacha20_block` para mensajes que superen los 64 bytes.
- Deberá manejar correctamente el bloque final parcial cuando la longitud del mensaje no sea múltiplo de 64 bytes.
- El programa deberá ejecutar los casos de prueba especificados y mostrar en pantalla los resultados: texto plano, flujo de clave en hexadecimal, texto cifrado en hexadecimal, y texto recuperado tras el descifrado.
- La clave y el nonce de prueba deben poder modificarse directamente en el código fuente.

3.3.3. Integración y compilación

- El proyecto deberá utilizar el entorno basado en Litex proveído para el desarrollo del proyecto, modificando el `main.c` para permitir la funcionalidad agregada.
- Deberá modificar los archivos fuente proveídos para que su implementación sea compilada y se pueda ejecutar en un procesador RISC-V dentro de los artefactos generados por Litex.

3.4. Casos de prueba

3.4.1. Casos de prueba obligatorios

Para validar el correcto funcionamiento de la implementación, se deberán ejecutar los siguientes casos de prueba:

- **Prueba 1 - Vector oficial RFC 8439:** Ejecutar la función de bloque con la clave formada por los bytes consecutivos `0x00` a `0x1f` (32 bytes), el nonce `0x00000009`, `0x0000004a`, `0x00000000` y contador igual a 1. Las primeras cuatro palabras del bloque de salida deben

ser exactamente 0xe4e7f110, 0x15593bd1, 0x1fdd0f50, 0xc47120a3, conforme al vector de prueba publicado en el RFC 8439. Este caso valida la corrección del núcleo del algoritmo contra el estándar internacional.

- **Prueba 2 - Ciclo completo de cifrado y descifrado:** Cifrar la cadena *Hola Mundo ChaCha20* con una clave y nonce definidos en el código fuente, y verificar que el descifrado del resultado produce exactamente la cadena original. Se deberá mostrar el texto plano, el texto cifrado en hexadecimal y el texto recuperado.
- **Prueba 3 - Mensaje de múltiples bloques:** Cifrar y descifrar un mensaje de longitud superior a 64 bytes para verificar el correcto incremento del contador entre bloques y la ausencia de corrupción en los límites de bloque. Se deberá demostrar que el descifrado recupera íntegramente el mensaje original.

3.5. Requisitos de implementación y validación

3.5.1. Implementación

El código deberá separar claramente las responsabilidades: la inicialización del estado, la gestión de bloques y la interfaz con los datos se implementará en C, mientras que las funciones críticas de generación de flujo de clave y cifrado XOR se implementarán en ensamblador RISC-V. Las funciones en ensamblador deberán seguir estrictamente las convenciones de llamada RISC-V y estar optimizadas para eficiencia en el uso de registros.

Los productos finales de esta etapa son:

1. El código fuente. Todo código debe ser compilable y ejecutable en el ambiente basado en Litex proveído.
2. Documentación técnica que incluya descripción y detalles de implementación de la solución, con diagramas, decisiones de diseño, resultados generales y análisis de rendimiento en formato *Markdown* (.md). Utilice el estilo de *blog post*/tutorial técnico como base para la documentación.

3.5.2. Validación

Para validar el diseño y correcto funcionamiento del algoritmo ChaCha20, cada grupo deberá ejecutar los casos de prueba especificados anteriormente y documentar los resultados. Adicionalmente, se deberá demostrar la capacidad de cifrar y descifrar mensajes de longitud variable, mostrando el correcto manejo del contador entre bloques y del bloque final parcial.

Salidas esperadas de esta etapa: Código fuente de programa de validación ejecutable que demuestre el correcto funcionamiento con los casos de prueba especificados.

4. Evaluación del proyecto

La evaluación del proyecto se da bajo los siguientes rubros:

- **Funcionalidad (70 %):** El profesor evaluará de manera independiente la solución analizando el código, así como ejecutando los scripts y aplicaciones de prueba del repositorio correspondiente.

Los aspectos que se revisarán **durante** la evaluación funcional son los siguientes:

1. Código fuente:
 - a) Funciones de generación de bloque de flujo de clave y cifrado XOR en ensamblador RISC-V.
 - b) Programa principal en C con gestión de estado e interfaz de usuario.
 - c) Sistema de compilación e integración.
 2. Validación:
 - a) Ejecución correcta de todos los casos de prueba.
 - b) Verificación contra el vector oficial del RFC 8439.
 - c) Demostración de cifrado y descifrado de mensajes de longitud variable.
- **Documentación de la implementación (20 %):** La documentación en formato *Markdown* (.md) debe contener al menos la siguiente información:
 1. Descripción de la arquitectura del software: separación entre capas C y ensamblador, interfaces utilizadas y justificación de las decisiones de diseño.
 2. Explicación de las funcionalidades implementadas.
 3. Breve discusión de resultados.
 - **Uso del repositorio (10 %):** El código fuente debe ser manejado con versiones a través de GitHub, en el repositorio creado para dicho fin.

5. Aspectos finales

En este proyecto, se deben tomar en cuenta las siguientes consideraciones:

- Se puede realizar en grupos de hasta máximo **tres** personas.
- Toda información adicional empleada debe tener su referencia respectiva.
- La fecha de revisión del proyecto será el miércoles 8 de abril de 2026, durante el periodo de la clase.
- El valor porcentual de este proyecto es de 10 % sobre la nota del curso.
- Una implementación ordenada y funcional es fundamental. Todos los artefactos del proyecto deben ser versionados, particularmente el código fuente de implementación, validados y reproducibles.
- Cualquier intento de plagio, será procesado de acuerdo con la normativa del ITCR.
- Puede hacer uso de herramientas de Inteligencia Artificial para ayudar en la implementación del proyecto. Sin embargo, se deben validar y entender los resultados obtenidos, que serán consultados durante la defensa del proyecto. Un fallo en la defensa, se colocará la nota de 0 en los rubros de implementación.